



AeroNAV System Report

Airline Flight Management System

Overview

The AeroNAV System is a robust and efficient flight airline management system designed to streamline flight operations for both users and administrators. Developed using Python, MySQL, and external APIs (AeroDataBox and OpenWeatherMap), the system provides a comprehensive set of functionalities, including user authentication, flight booking, cancellation, and viewing for regular users, as well as flight creation, deletion, and delay management for administrators. A standout feature is its advanced flight route optimization, which accounts for fuel efficiency, travel time, and wind conditions to determine optimal flight paths, whether direct or via connecting hubs. The system is modular, with distinct components for authentication, user actions, admin operations, and route optimization, ensuring maintainability and scalability.

System Components

The AeroNAV System is implemented across five Python files, each handling specific aspects of the system:

1. auth.py: Manages user authentication and database connectivity.

- Functions:
 - `get_connection()`: Establishes a connection to the MySQL database 'flights'.
 - `register(username, password, role="user")`: Registers a new user with a securely hashed password using bcrypt.
 - `login(username, password)`: Authenticates users by verifying credentials against the 'users' table.

- Key Features: Secure password hashing, role-based access control (user/admin), and reliable database connectivity.

2. `user_actions.py`: Handles user interactions with the flight system.

- Functions:
 - `view_active_flights()`: Retrieves and displays all flights with status='Active' from the 'flights_registered' table.
 - `book_flight(user_id, flight_id)`: Inserts a booking record into the 'flight_bookings' table.
 - `cancel_booking(user_id, booking_id)`: Deletes a booking from the 'flight_bookings' table, ensuring user-specific access.
 - `view_my_bookings(user_id)`: Displays a user's bookings with details joined from 'flights_registered'.
- Key Features: Intuitive user interface for flight management, secure booking and cancellation processes tied to user IDs.

3. `admin_actions.py`: Provides administrative tools for managing flights.

- Functions:
 - `create_flight(origin, dest, hubs, departure_time, passng=150)`: Creates optimized flights (direct or via hubs) using the `optimize_with_connection` function and saves them to the database.
 - `save_flight(res, departure_time, passng)`: Inserts flight details into the 'flights_registered' table.
 - `delete_flight(flight_id)`: Removes a specified flight from the database.
 - `delay_flight(flight_id, delay_minutes)`: Updates a flight's status to "Delayed" and adjusts its departure time using SQL's `DATE_ADD`.
- Key Features: Integration with route optimization, robust error handling, and seamless database updates.

4. flight_optimizer.py: Implements sophisticated flight route optimization and visualization.

- Functions:

- get_airport_coords(icao): Fetches airport latitude and longitude using the AeroDataBox API.

- haversine_distance(lat1, lon1, lat2, lon2): Calculates the great-circle distance between two points in nautical miles.

- calculate_bearing(lat1, lon1, lat2, lon2): Computes the initial heading between two geographic points.

- midpoint_coords(lat1, lon1, lat2, lon2): Determines the midpoint of a great-circle path.

- get_wind_data(lat, lon): Retrieves wind speed and direction from the OpenWeatherMap API.

- flight_traffic_on_route(origin_icao, dest_icao, custom_time): Counts active flights on a route using the AeroDataBox API.

- optimize_flight_path(origin_icao, dest_icao, custom_time, passng): Optimizes a flight leg for fuel efficiency and time, factoring in wind conditions and aircraft capabilities.

- optimize_with_connection(origin, dest, hubs, custom_time, passng): Compares direct and connecting flights to select the optimal route based on fuel and time.

- plot_route(routes): Visualizes flight routes on a world map using matplotlib and cartopy.

- Key Features: Advanced route optimization considering fuel, time, and wind; geographic visualization of flight paths.

5. main.py: Serves as the entry point, providing a command-line interface for user and admin interactions.

- Functions:

- user_menu(user): Displays a menu for users to view flights, book, cancel, or view bookings.

- admin_menu(user): Provides admin options to create, delete, or delay flights.

- Key Features: User-friendly command-line interface, role-based menu navigation, and integration with other modules.

Technical Details

- Database: The system uses a MySQL database named 'flights' with tables:
 - users: Stores user credentials (username, password_hash, role).
 - flights_registered: Contains flight details (flight_id, flight_number, airline, origin_icao, destination_icao, departure_time, arrival_time, status, aircraft_type, passengers).
 - flight_bookings: Manages user bookings (booking_id, user_id, flight_id).
 - airplanes: Stores aircraft data (type, airspeed_kts, fuel_rate_kg_h, passengers).
- External APIs:
 - AeroDataBox: Provides airport coordinates and flight traffic data.
 - OpenWeatherMap: Supplies wind speed and direction for route optimization.
- Libraries:
 - mysql-connector-python: For database interactions.
 - bcrypt: For secure password hashing.
 - requests: For API calls.
 - matplotlib and cartopy: For route visualization.
 - Standard libraries: math, datetime.
- Optimization Logic: The system optimizes flight routes by:
 - Calculating great-circle distances and bearings.
 - Incorporating wind data to adjust ground speed.
 - Selecting aircraft from the 'airplanes' table based on passenger capacity, fuel efficiency, and speed.
 - Comparing direct and connecting routes, factoring in a 1-hour layover for connections.

Execution Flow

1. **Program Start:** The main.py script prompts for login credentials and calls login from auth.py to authenticate users.
2. **Authentication:** Verifies username and password against the 'users' table, returning a user object with user_id, username, and role.
3. **Menu Navigation:**
 - **Users:** Access the user_menu to view active flights, book/cancel flights, or view bookings.
 - **Admins:** Access the admin_menu to create, delete, or delay flights.
4. **Flight Creation:** Admins input origin, destination, hubs, departure time, and passengers. The create_flight function uses optimize_with_connection to determine the best route and saves flights to the database.
5. **Optimization:** The flight_optimizer.py module fetches coordinates, wind data, and traffic, then selects the best aircraft and route based on fuel and time metrics.
6. **Database Operations:** All modules interact with the MySQL database for persistent storage and retrieval.
7. **Visualization:** The plot_route function (though not called in the main flow) can generate a map of flight routes.

Limitations and Potential Improvements

1. **API Security:** Hardcoded API keys in flight_optimizer.py pose a security risk. Keys should be stored in environment variables.
2. **Error Handling:** While robust, additional validation (e.g., ICAO code format, passenger count limits) could enhance reliability.
3. **Input Validation:** Minimal checks exist for user inputs in main.py (e.g., departure time format, ICAO codes), risking crashes or incorrect data.

4. **Database Connection Management:** Lack of connection pooling or retry logic may lead to performance issues or failures during high load.
5. **User Interface:** The command-line interface limits usability; upgrading to a web or graphical interface would improve user experience.
6. **Flight Traffic Utilization:** The `flight_traffic_on_route` data is collected but not used in optimization, missing an opportunity to avoid congested routes.
7. **Route Visualization:** The `view_active_flights` function redundantly fetches coordinates and lacks robust error handling, potentially disrupting plotting.
8. **Security Credentials:** Hardcoded database credentials in `auth.py` are a security vulnerability; they should be moved to environment variables.
9. **Database Schema:** No setup script or validation exists for database tables, risking issues if tables are missing or misconfigured.
10. **Logging:** Reliance on print statements instead of structured logging hinders debugging and monitoring in production.
11. **Time Zone Support:** The system uses UTC implicitly without configurable time zones, limiting flexibility for global operations.

Acknowledgments

The development of the AeroNAV System was significantly enhanced by the contributions of Pranav and the team, whose expertise and dedication in coding were instrumental in bringing this project to fruition. Their efforts in implementing key functionalities, debugging, and ensuring system reliability are deeply appreciated.

Conclusion

The AeroNAV System is a powerful tool for managing airline operations, offering user-friendly flight management and sophisticated route optimization. Its modular design and integration with external APIs make it a scalable solution for modern aviation needs. With minor improvements, it can serve as a robust platform for both small and large-scale airline operations.

Date: September 16, 2025